00 PROFESSIONAL-GRADE NESTJS PROJECT STRUCTURE - DETAILED EXPLANATION

```
my-app/
├── src/
│   ├── main.ts
│   ├── app.module.ts
│   │
│   ├── config/                    # env validation, config loading (@nestjs/config)
│   │   ├── configuration.ts
│   │   └── validation.schema.ts
│   │
│   ├── common/                     # shared, cross-cutting code
│   │   ├── decorators/
│   │   ├── filters/                # exception filters
│   │   ├── guards/                 # auth guards, roles guards
│   │   ├── interceptors/           # logging, transform, timeout
│   │   ├── pipes/                  # validation pipes
│   │   ├── middleware/
│   │   └── constants/
│   │
│   ├── database/
│   │   ├── database.module.ts
│   │   ├── migrations/
│   │   └── seeds/
│   │
│   ├── modules/                    # feature modules (domain-driven)
│   │   ├── auth/
│   │   │   ├── auth.module.ts
│   │   │   ├── auth.controller.ts
│   │   │   ├── auth.service.ts
│   │   │   ├── strategies/         # jwt.strategy.ts, local.strategy.ts
│   │   │   ├── guards/
│   │   │   └── dto/
│   │   │
│   │   ├── users/
│   │   │   ├── users.module.ts
│   │   │   ├── users.controller.ts
│   │   │   ├── users.service.ts
│   │   │   ├── entities/ (or schemas/)
│   │   │   ├── dto/
│   │   │   └── users.controller.spec.ts
│   │   │
│   │   └── ...other domains (orders, products, etc.)
│   │
│   ├── shared/                     # shared services (mailer, cache, storage)
│   │   ├── logger/
│   │   └── redis/
│   │
│   └── health/                     # health checks (@nestjs/terminus)
│
├── test/
│   ├── e2e/
│   └── jest-e2e.json
│
```

```
├── docker/
│   ├── Dockerfile
│   └── docker-compose.yml
│
├── .env
├── .env.example
├── nest-cli.json
├── package.json
└── tsconfig.json
```

This document describes a recommended folder/file structure for a large, production-grade NestJS application, along with what each piece is for, why it exists, and when you'd add to it.

NOTE This is a TARGET structure, not what currently exists in this repo.

The current repo is still the default Nest CLI scaffold (main.ts, app.module.ts, app.controller.ts, app.service.ts only). This file is a reference for how to grow the project as features are added.

TOP-LEVEL LAYOUT

```
my-app/
├── src/                    Application source code
├── test/                   End-to-end (e2e) tests
├── docker/                 Containerization files
├── .env                    Local environment variables (NEVER committed)
├── .env.example            Template listing required env vars (committed)
├── nest-cli.json           Nest CLI configuration (compiler options, assets)
├── package.json            Dependencies and npm scripts
└── tsconfig.json           TypeScript compiler configuration
```

src/main.ts

The application entry point. Responsibilities:

```
  - Creates the Nest application instance (NestFactory.create).
  - Applies global middleware: ValidationPipe, exception filters,
    interceptors, CORS config, Helmet, compression.
  - Sets up Swagger/OpenAPI docs if used.
  - Starts the HTTP listener on the configured port.
Keep this file thin — it wires things up, it doesn't contain business logic.
```

src/app.module.ts

The root module. It imports:

- ConfigModule (global config) - DatabaseModule (DB connection) - All feature modules (AuthModule, UsersModule, etc.) - Global guards/interceptors registered via APP_GUARD / APP_INTERCEPTOR providers. This file should stay small — it's just composition, not implementation.

src/config/

Centralizes environment/configuration handling instead of scattering `process.env.X` calls throughout the codebase.

```
config/configuration.ts
  A function that reads process.env and returns a typed, structured
  config object (e.g. { database: {...}, jwt: {...}, port: ... }).
  Registered with @nestjs/config's ConfigModule.forRoot({ load: [...] }).
```

config/validation.schema.ts A Joi (or class-validator/zod) schema that validates required env vars exist and have the right shape at startup. Fails fast with a clear error instead of a confusing runtime crash later.

Why this matters: as an app grows, config sprawl becomes a real source of bugs (typo'd env var names, missing required vars in prod). Centralizing and validating config catches these issues at boot time.

src/common/

Cross-cutting code used across multiple feature modules. Nothing here is specific to one domain (users, orders, etc.) — it's framework-level plumbing.

```
common/decorators/
  Custom parameter/method decorators, e.g. @CurrentUser() to pull the
  authenticated user off the request, @Roles('admin') for RBAC metadata.
```

common/filters/ Exception filters (implements ExceptionFilter). Catch errors thrown anywhere in the app and transform them into consistent HTTP responses (e.g. a global AllExceptionsFilter that formats all errors the same way and logs them).

```
common/guards/
  Route guards (implements CanActivate). E.g. JwtAuthGuard checks a valid
  token is present; RolesGuard checks the user has permission for the
  route. Guards run before the route handler.

common/interceptors/
  Wrap the request/response pipeline (implements NestInterceptor).
  Common uses: LoggingInterceptor (logs request duration),
  TransformInterceptor (wraps all responses in a consistent envelope
  like { data, meta }), TimeoutInterceptor (kills slow requests).
```

common/pipes/ Data transformation/validation applied to route arguments. Usually you rely on the built-in ValidationPipe with class-validator DTOs, but custom pipes go here (e.g. ParseObjectIdPipe for Mongo IDs).

```
common/middleware/
  Express/Fastify-level middleware (implements NestMiddleware), run
  before Nest's routing layer. Use for things like request-id injection,
  raw body capture for webhooks, etc.

common/constants/
  Shared enums, magic strings, injection tokens (e.g. CACHE_TTL,
  ROLES_KEY) used across modules.
```

src/database/

database/database.module.ts Sets up the ORM connection (TypeORM, Prisma, Mongoose, etc.) using values from ConfigService. Exported so feature modules can inject repositories/connections.

database/migrations/ Versioned schema changes (if using TypeORM/Prisma migrations). Ensures schema changes are tracked, reviewable, and repeatable across environments instead of relying on synchronize:true in production.

database/seeds/ Scripts to populate the DB with initial or test data (e.g. an admin user, reference/lookup tables).

src/modules/ (feature modules — the heart of the app)

Each business domain gets its own self-contained module. This is the "domain-driven" or "feature-first" organization style, as opposed to grouping by technical layer (all controllers together, all services together) which doesn't scale past a handful of endpoints.

Example: src/modules/users/

```
users.module.ts        Declares controller, service, providers, exports.
users.controller.ts    HTTP layer only — routes, request/response shape,
                        delegates all logic to the service.
users.service.ts       Business logic. Talks to the repository/database,
                        enforces business rules, throws domain errors.
entities/               ORM entity/schema definitions (e.g. user.entity.ts)
                        describing the DB table shape.
dto/                    Data Transfer Objects — class-validator-decorated
                        classes describing the shape of incoming requests
                        (create-user.dto.ts, update-user.dto.ts) and
                        sometimes outgoing responses. These are what
                        ValidationPipe validates against.
users.controller.spec.ts / users.service.spec.ts
                        Unit tests colocated with the code they test.
```

Why colocate everything by feature: when you need to change or remove the "users" feature, everything related lives in one folder. It also makes it obvious what a module's public surface is (what it exports) vs. private internals.

Other typical feature modules: auth/, orders/, products/, payments/, notifications/ — one per bounded business domain.

```
modules/auth/ (special case, called out separately because nearly every
app has one and it has extra structure)
  auth.module.ts / auth.controller.ts / auth.service.ts — login, refresh,
    logout endpoints and logic.
  strategies/           Passport strategies (jwt.strategy.ts validates
                        bearer tokens, local.strategy.ts validates
                        username/password on login).
  guards/               AuthGuard('jwt'), AuthGuard('local') wrappers,
                        plus any auth-specific guards not generic enough
                        for common/guards/.
  dto/                  login.dto.ts, register.dto.ts, etc.
```

src/shared/

Reusable *services* (not pure cross-cutting plumbing like common/) that multiple feature modules depend on as collaborators.

```
shared/logger/     A structured logger (e.g. wrapping Winston/Pino),
                    injectable app-wide, replacing console.log with
                    leveled, structured, environment-aware logging.
shared/redis/      Cache/session client setup, injectable service for
                    get/set/expire operations used by multiple modules.
```

The distinction from common/: common/ is framework wiring (guards, filters, interceptors) that Nest calls into automatically; shared/ is business- adjacent utility services that modules explicitly inject and call.

src/health/

Health check endpoint(s) using @nestjs/terminus — reports DB connectivity, disk space, memory usage, external service reachability. Used by load balancers/orchestrators (Kubernetes liveness/readiness probes, AWS ALB health checks) to know if the instance is healthy.

test/

test/e2e/	Full end-to-end tests that spin up the whole Nest app (via Test.createTestingModule) and hit real HTTP routes with supertest, verifying full request/ response behavior including guards, pipes, DB.
test/jest-e2e.json	Jest config specific to e2e tests (separate from unit test config in package.json), usually with a longer timeout and different test match pattern.

Unit tests (*.spec.ts) are NOT here — they live next to the code they test inside src/, per module, so they're easy to find and keep in sync.

docker/

```
Dockerfile          Multi-stage build: install deps, build TypeScript,
                    copy only the compiled dist/ + node_modules into a
                    slim final image (e.g. node:alpine) for production.
docker-compose.yml  Local dev orchestration — spins up the app plus its
                    dependencies (Postgres, Redis, etc.) together so a
                    new developer can run one command to get a full
                    working environment.
```

.env / .env.example

.env	Actual secrets/config for the local machine. Must be in .gitignore — never committed.
.env.example	Same keys as .env but with placeholder/dummy values. Committed to the repo so any developer knows exactly which environment variables the app needs to run.

nest-cli.json / tsconfig.json / package.json

```
nest-cli.json    Tells the Nest CLI how to build the project (entry file,
                 whether to copy non-ts assets to dist/, etc.).
tsconfig.json    TypeScript compiler options (target, decorators,
                 strictness settings). tsconfig.build.json extends it
                 and excludes test files from production builds.
package.json     Dependency list and npm scripts (start, start:dev,
                 build, test, test:e2e, lint).
```

GUIDING PRINCIPLES BEHIND THIS LAYOUT

1. Feature-first, not layer-first. Group by business domain (users/, orders/) rather than by technical role (all controllers/, all services/). Scales much better once you have more than ~5 endpoints.

2. Controllers stay thin. Controllers only handle HTTP concerns (routes, status codes, request/ response shape). All business logic lives in services, so it can be unit tested without spinning up HTTP.

3. Validate at the boundary. DTOs + ValidationPipe validate everything coming in from outside the app. Once past that boundary, code can trust the data shape.

4. Cross-cutting concerns are declared, not scattered. Guards, filters, interceptors, pipes are registered centrally (often globally via APP_GUARD/APP_FILTER/APP_INTERCEPTOR providers in app.module.ts) instead of manually re-applied in every controller.

5. Config is centralized and validated at startup. Fail fast on misconfiguration rather than discovering a missing env var when a request happens to hit that code path in production.

6. Tests live close to what they test. Unit tests sit next to their source file; only true end-to-end tests live in test/.